Data analysis pipelines with pipeflow : : **CHEAT SHEET**

Basics

pipeflow builds data analysis pipelines from plain R functions. Each function becomes a **step**, and function parameters can be connected to other steps:

```
pip_add(p, "squared",
    \(x = ~numbers, k = 2) x^k)
```

Add step **squared** to pipeline `p`. Input parameter **x** is the output of step **numbers**, and **k** is a normal parameter.

```
p <- pip_new("demo") |>
   pip_add("numbers", \(n = 5) seq_len(n)) |>
   pip_add("squared", \(x = ~numbers, k = 2) x^k) |>
   pip_add("total", \(x = ~squared) sum(x))

pip_run(p)         # run all steps
pip_collect(p)     # collect step outputs
```

`print(p)` shows the pipeline as a table, one row per step:

step	params	depends	state	out
numbers	n		done	1,2,3,4,5
squared	x,k	numbers	done	1, 4, 9,16,25
total	x	squared	done	55

Build a pipeline

`pip_new("demo")`
create an empty pipeline.

`pip_add(p, "numbers", \(n = 5) seq_len(n))`
add a step at the end.

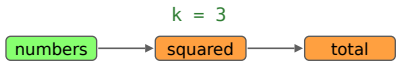
`pip_add(p, "fit", fun, tags = c("model", "core"))`
tag steps to select or collect them later.

Parameters

Arguments that are not inputs are parameters.

`pip_get_params(p)`
all parameters as a named list.

`pip_set_params(p, list(k = 3))`
set a parameter in all steps that have it. These steps and the steps that depend on them become outdated.



Run

Each step has a state:

- new — not run yet
- done — up to date, skipped by the next run
- outdated — an input or a parameter changed
- failed — the last run raised an error

`pip_run(p)`
run all steps where state is not yet done.

`pip_run(p, force = TRUE)`
force re-run of all steps regardless of state.

`pip_run(p["total"])`
run selected step(s), plus the upstream steps they need.

`pip_run(p, lgr = NULL)`
run without log messages.

Collect and inspect

`pip_collect(p)`
the outputs of all steps as a named list.

`pip_collect(p, by = "tags")`
grouped by tag.

`p[, c("step", "out")]`
output in table format.

`p[["total", "out"]]`
output of one step.

Views

Views are a window onto the pipeline by reference (no copy); most `pip_*()` functions work with views.

`v <- pip_view(p, tags = "core", state = "done")`
filter by certain columns of the step table.

`pip_set_params(v, list(k = 3))`
set parameters for all steps of the view.

`pip_run(v)`
run the steps of the view, plus the upstream steps they need.

`pip_collect(v)`
collect from the filtered view.

Select steps with [ ]

Creates views via subsetting.



`v <- p[2:3], p[c("squared", "total")]`
by position or name: the result is a view.



`v <- p[state == "outdated"]`
by an expression on the step table, as in data.table.

OPERATORS FROM DATA.TABLE

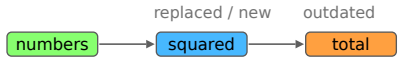
`%like%` `%ilike%` `%notin%` `%between%`

For example, `p[step %like% "^sq"]` selects step squared.

Modify steps

Changes are checked; affected steps run again:

`pip_replace(p, "squared", \(x = ~numbers) x^3)`
a new function for a step: the step becomes new, its downstream steps outdated.



`pip_rename(p, "total", "sum")`
rename a step and all references to it.

`pip_remove(p, "sum")`
remove a step (`force = TRUE`: also the steps that use it).

`pip_lock(p["numbers"]), pip_unlock(p)`
locked steps freeze their parameters and output: runs skip them.

Combine pipelines



`p <- rbind(p1, p2)`
a new pipeline with the steps of both. Clashing step names get a suffix: `data` becomes `data2`.



`pip_replace(p, "data2", \(x = ~prep) x)`
connects data2 with prep manually after the bind.

To connect the parts in one go, use:
`p <- rbind(p1, p2, bind = list(data = ~prep))`

Advanced features and workflows

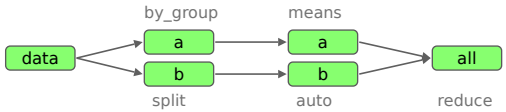
Ways to connect steps

`\(x = ~numbers)`
`x` is the output of step `numbers`.

`\(x = ~-1)`
the output of the previous step (`~-2`: two steps back).

`\(x = ~numbers, y = ~squared)`
several inputs.

Split · map · reduce



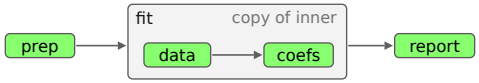

```
pip_add(p, "by_group", \(d = ~data) split(d$x, d$g),
        exec = "split")
```
`"split"`: the output is a named list of partitions.

```
pip_add(p, "means", \(x = ~by_group) mean(x))
```
`"auto"` (the default): runs once per partition.

```
pip_add(p, "all", \(m = ~means) unlist(m),
        exec = "reduce")
```
`"reduce"`: one call that gets all partitions as a named list.

Nested pipelines

A step can run a copy of another pipeline:




```
f <- as.function(inner, out = "coefs")
```
a function that runs a copy of `inner`. Its arguments are the unbound params of `inner`; `out` is the step output it returns.

```
pip_add(outer, "fit",
        as.function(inner, out = "coefs",
                    bind = list(data = ~prep)))
```
as a step: `bind` connects the param `data` to step `prep`. The other unbound params of `inner` become params of `fit`.

Copy and save

p → ▦ q → ▦ `q <- pip_clone(p)`
an independent copy. After `q <- p`, both names refer to the same pipeline.

`saveRDS(p, "p.rds")`, `p <- readRDS("p.rds")`
save and restore a pipeline with its outputs and states.

Handle failures




```
pip_run(p, on_error = "continue")
```
by default, a run stops at the first error. With `"continue"`, a failing step gets state failed, the steps that use it are skipped, and all others run.

```
pip_add(p, "safe", \(m = ~try(model)) {
    if (inherits(m, "pipeflow_failure")) NA else m
})
```
`~try()`: run even if `model` failed; `m` is then a `pipeflow_failure`.

```
pip_run(p)
```
once fixed, the next run picks up the failed and skipped steps.

RUN STATE

`pip_run_state(p)` tells how the last run ended:

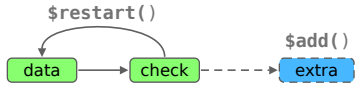
ready	without failures
failed	stopped at an error
continued	went on after failures
halted	a step called `.self$halt()`

Good to know

- `pip_*()` functions change the pipeline in place and return it invisibly, so they can be chained with `|>`.
- `p$add(...)` is short for `pip_add(p, ...)`; likewise `$run()`, `$set_params()`, `$collect()`, ...
- Every argument of a step function needs a default. Primitives like `sum` have none: wrap them, `\(x = ~-1) sum(x)`.

Self-modifying pipelines

In a step function, `.self` is the running pipeline.



`.self$add()`, `$replace()`, `$remove()`, ...
change the pipeline during a run.

`.self$restart()`
when the step is done, run again with the new structure.




```
pip_add(p, "check", \(x = ~data) {
    if (length(x) == 0) .self$halt()
    x
})
```
`.self$halt()`: stop the run after this step; the next `pip_run()` continues.

Non-trivial parameter values

pipeflow supports non-trivial parameter values.

```
my_param <- structure(
    list(value = 1.23, label = "My label"),
    class = "foo"
)
p <- pip_new("demo") |>
    pip_add("step", \(x = my_param) x)

# getter for class foo
pip_param_value.foo <- function(x, ...) x$value

# setter for class foo
`pip_param_value<-.foo` <- function(x, ..., value) {
    if (inherits(value, "foo")) return(value)
    x$value <- value
    x
}
```
register getter and setter methods.

`pip_run(p)`
the step function receives `1.23`, not the `foo` object.

`pip_get_params(p)`
returns the stored `foo` object.

`pip_set_params(p, list(x = 99))`
sets the value to `99`; the label is kept.